In [2]:
```
# %load_ext cuml.accel
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans, DBSCAN
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.ensemble import RandomForestClassifier
from xgboost import XGBClassifier
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from imblearn.over_sampling import RandomOverSampler
from sklearn.metrics import confusion_matrix, classification_report
import plotly.express as px
import warnings
warnings.filterwarnings('ignore')
```

In [3]:
```
def draw_confusion_matrix(cm, title='Confusion Matrix'):
    plt.figure(figsize=(6, 4))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=False,
                xticklabels=['Not Fraud', 'Fraud'], yticklabels=['Not Fraud', 'Frau
    plt.title(title)
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.show()
```

In [4]:
```
import kagglehub

path = kagglehub.dataset_download("ealaxi/banksim1")
print("Path to dataset files:", path)
```

```
Path to dataset files: /kaggle/input/banksim1
```

In [5]:
```
data = pd.read_csv(path+'/bs140513_032310.csv')
```

In [6]:
```
data = data.drop(['zipcodeOri', 'zipMerchant'], axis=1)
data['customer'] = data['customer'].str[1:-1]
data['age'] = data['age'].str[1:-1]
data['gender'] = data['gender'].str[1:-1]
data['merchant'] = data['merchant'].str[1:-1]
data['category'] = data['category'].str.replace('es_', '')
data = data[data['gender'] != 'U']
data['age'] = data['age'].replace('U', '7')
data['age'] = data['age'].astype(float)
```

In [7]:
```
def create_amount_thresh(amount):
    if amount <= 500:
        return '0-500'
    elif amount <= 1000:
        return '500-1000'
    elif amount <= 1500:
        return '1000-1500'
```

```
    elif amount <= 2000:
        return '1500-2000'
    elif amount <= 2500:
        return '2000-2500'
    elif amount <= 3000:
        return '2500-3000'
    else:
        return '>3000'
```

In [8]:
```
data['amount_thresh'] = data['amount'].apply(create_amount_thresh)
```

In [9]:
```
customer = data.groupby('customer').size().reset_index(name='customer_total_trans')
customer['customer_ID'] = range(1, len(customer) + 1)

merchant = data.groupby('merchant').size().reset_index(name='merchant_total_trans')
merchant['merchant_ID'] = range(1, len(merchant) + 1)

category = data.groupby('category').size().reset_index(name='category_total_trans')
category['category_ID'] = range(1, len(category) + 1)

amount_thresh = data.groupby('amount_thresh').size().reset_index(name='amount_thres
amount_thresh['amount_thresh_ID'] = range(1, len(amount_thresh) + 1)
```

In [10]:
```
data = data.merge(customer, on='customer').merge(merchant, on='merchant')
data = data.drop(['customer', 'merchant'], axis=1)
data = data.merge(category, on='category').merge(amount_thresh, on='amount_thresh')
data['category'] = data['category_ID']
data['amount_thresh'] = data['amount_thresh_ID']
data = data.drop(['category_ID', 'amount_thresh_ID'], axis=1)
```

In [11]:
```
age = data.groupby('age').size().reset_index(name='age_total_trans')
gender = data.groupby('gender').size().reset_index(name='gender_total_trans')
data = data.merge(age, on='age').merge(gender, on='gender')
```

In [12]:
```
total_freq = len(data)
data['merchant_total_trans'] = (data['merchant_total_trans'] / total_freq * 100).ro
data['category_total_trans'] = (data['category_total_trans'] / total_freq * 100).ro
data['amount_thresh_total_trans'] = (data['amount_thresh_total_trans'] / total_freq
data['age_total_trans'] = (data['age_total_trans'] / total_freq * 100).round(5)
data['gender_total_trans'] = (data['gender_total_trans'] / total_freq * 100).round(
```

In [13]:
```
data = data.drop('step', axis=1)
data = data[['fraud'] + [col for col in data.columns if col != 'fraud']]
```

In [14]:
```
data = pd.get_dummies(data, columns=['age', 'gender', 'category', 'amount_thresh'],
```

In [15]:
```
data['fraud'] = data['fraud'].map({1: 'F', 0: 'NF'})
```

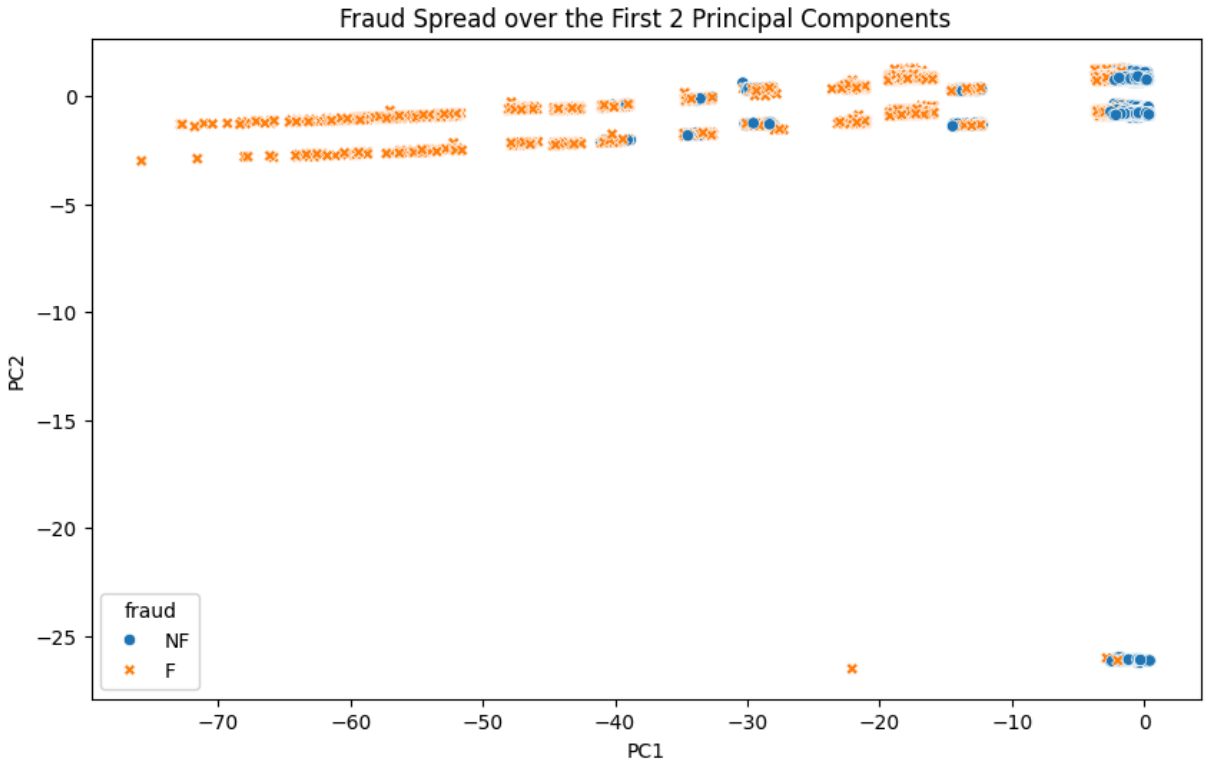
In [16]:
```
print("Data dimensions:", data.shape)
```

```
Data dimensions: (594128, 43)
```

In [17]:
```
pca_cols = ['age_0.0', 'age_1.0', 'age_2.0', 'age_3.0', 'age_4.0', 'age_5.0', 'age_
            'gender_E', 'gender_F', 'gender_M',
```

```
            'category_1', 'category_2', 'category_3', 'category_4', 'category_5', '
            'category_8', 'category_9', 'category_10', 'category_11', 'category_12'
            'amount', 'amount_thresh_1', 'amount_thresh_2', 'amount_thresh_3', 'amo
            'amount_thresh_6', 'amount_thresh_7']
scaler = StandardScaler()
pca_data_scaled = scaler.fit_transform(data[pca_cols])
pca = PCA(n_components=2)
pca_data = pca.fit_transform(pca_data_scaled)
pca_df = pd.DataFrame(pca_data, columns=['PC1', 'PC2'])
pca_df['fraud'] = data['fraud']
```

In [18]:
```
plt.figure(figsize=(10, 6))
sns.scatterplot(x='PC1', y='PC2', hue='fraud', style='fraud', data=pca_df)
plt.title('Fraud Spread over the First 2 Principal Components')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.show()
```



In [19]:
```
print("Explained variance ratio:", pca.explained_variance_ratio_)
```

```
Explained variance ratio: [0.09923016 0.06098442]
```

In [20]:
```
fraud_data = data[data['fraud'] == 'F']
non_fraud_data = data[data['fraud'] == 'NF']
```

In [21]:
```
non_fraud_sample = non_fraud_data.sample(frac=0.0166, random_state=123)  # Approx 9
undersampling_data = pd.concat([non_fraud_sample, fraud_data]).sample(frac=1, rando
print("Undersampling data dimensions:", undersampling_data.shape)
```

```
Undersampling data dimensions: (16943, 43)
```

In [22]:
```
X = undersampling_data.drop('fraud', axis=1)
y = undersampling_data['fraud']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_st

# Convert string labels to numerical labels
y_train = y_train.map({'F': 1, 'NF': 0})
y_test = y_test.map({'F': 1, 'NF': 0})
```

In [23]:
```
rf_model = RandomForestClassifier(random_state=123)
xgb_model = XGBClassifier(random_state=123, use_label_encoder=False, eval_metric='l
```

In [24]:
```
rf_model.fit(X_train, y_train)
xgb_model.fit(X_train, y_train)
```

Out[24]:

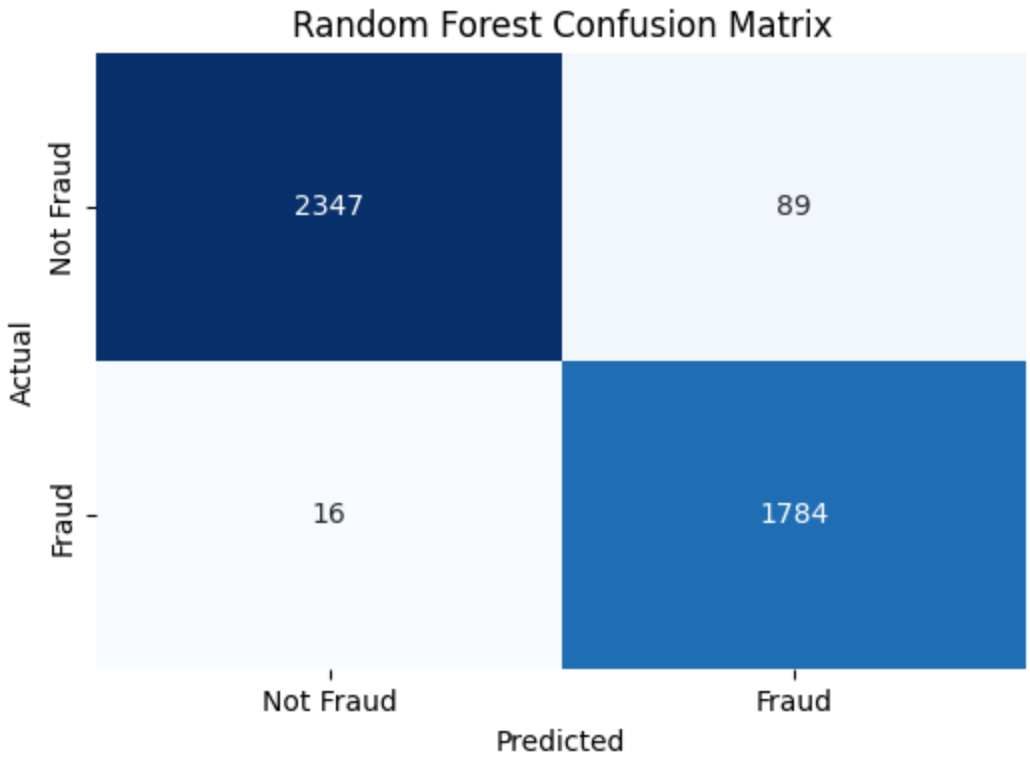
XGBClassifier

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=No
ne,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
```

In [25]:
```
rf_pred = rf_model.predict(X_test)
xgb_pred = xgb_model.predict(X_test)
```

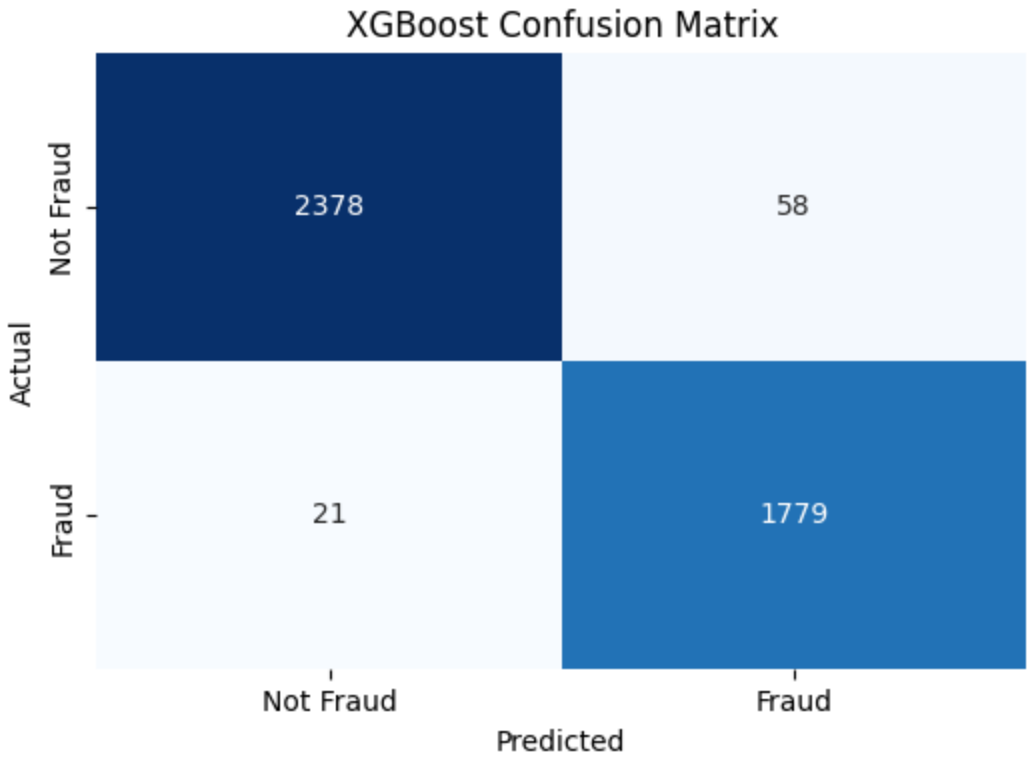
In [26]:
```
print("Random Forest Confusion Matrix:")
cm_rf = confusion_matrix(y_test, rf_pred, labels=[0, 1])
draw_confusion_matrix(cm_rf, title='Random Forest Confusion Matrix')
```

```
Random Forest Confusion Matrix:
```



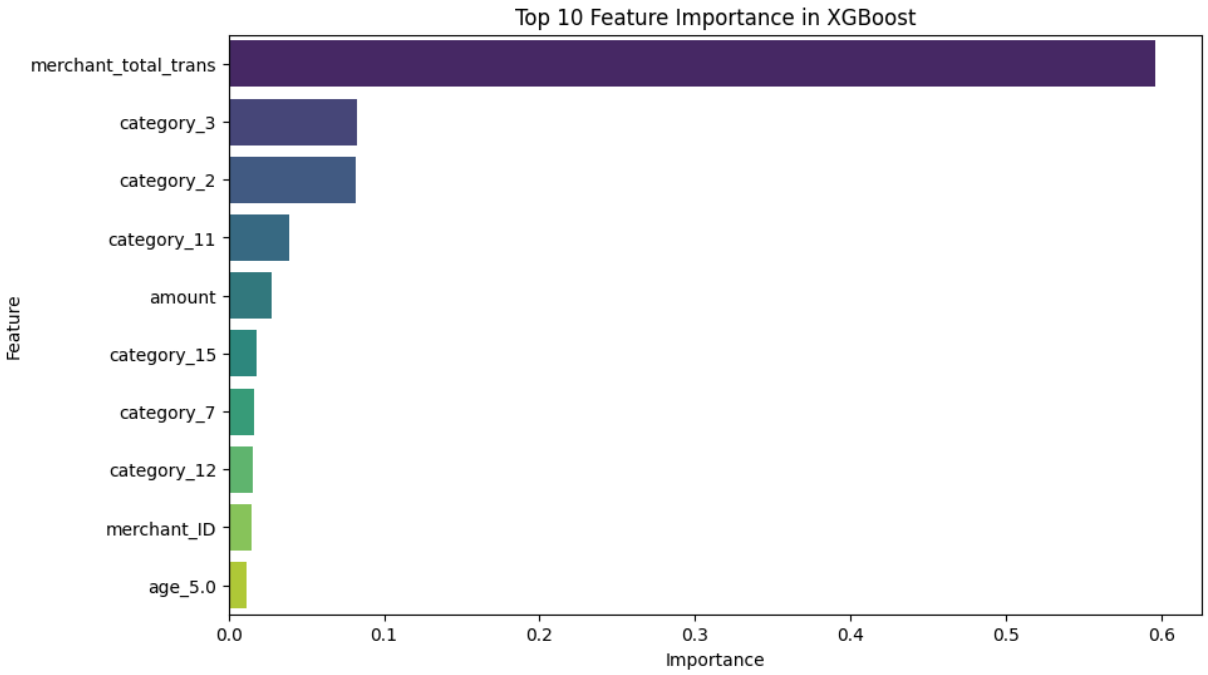
In [27]:
```
print("XGBoost Confusion Matrix:")
cm_xgb = confusion_matrix(y_test, xgb_pred, labels=[0,1])
draw_confusion_matrix(cm_xgb, title='XGBoost Confusion Matrix')
```

```
XGBoost Confusion Matrix:
```



In [28]:
```
feature_importance = pd.Series(xgb_model.feature_importances_, index=X_train.column
plt.figure(figsize=(10, 6))
sns.barplot(x=feature_importance.values, y=feature_importance.index, palette='virid
```

```
plt.title('Top 10 Feature Importance in XGBoost')
plt.xlabel('Importance')
plt.ylabel('Feature')
plt.show()
```



In [29]:
```
ros = RandomOverSampler(sampling_strategy='auto', random_state=123)
X_ros, y_ros = ros.fit_resample(data.drop('fraud', axis=1), data['fraud'])
```

In [30]:
```
X_train_ros, X_test_ros, y_train_ros, y_test_ros = train_test_split(X_ros, y_ros, t
```

In [32]:
```
rf_model_ros = RandomForestClassifier(random_state=123)
xgb_model_ros = XGBClassifier(random_state=123, use_label_encoder=False, eval_metri

# Convert string labels to numerical labels for oversampled data
y_train_ros = y_train_ros.map({'F': 1, 'NF': 0})
y_test_ros = y_test_ros.map({'F': 1, 'NF': 0})

rf_model_ros.fit(X_train_ros, y_train_ros)
xgb_model_ros.fit(X_train_ros, y_train_ros)
```

Out[32]:

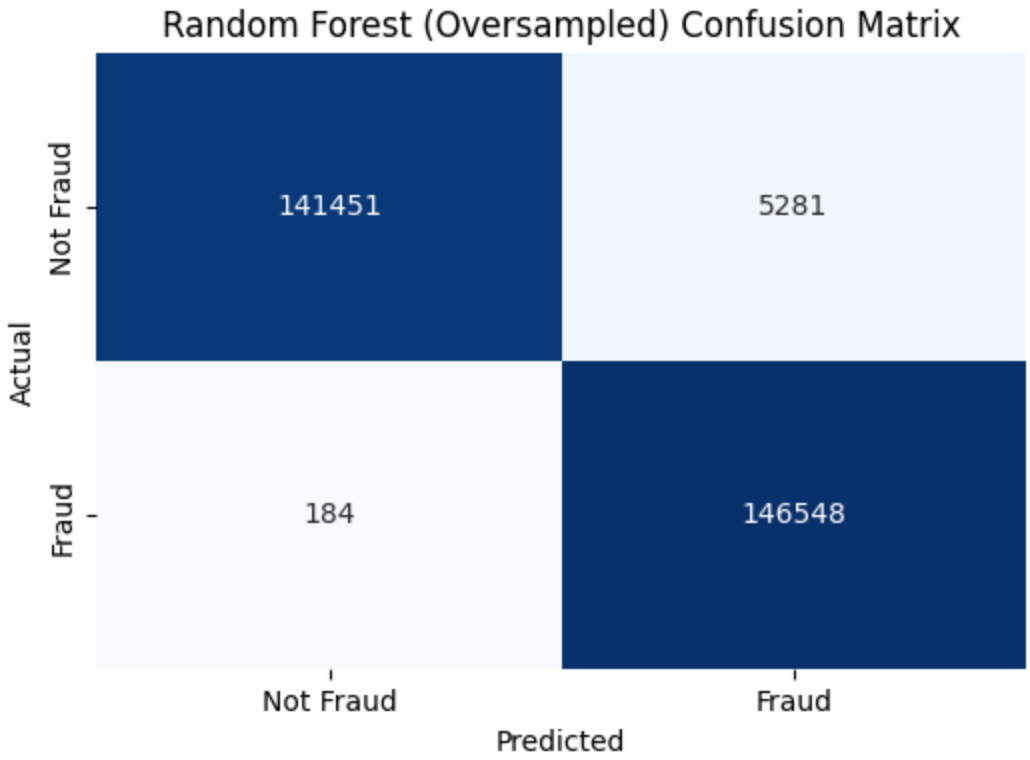
XGBClassifier

```
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=No
ne,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
```

In [33]:
```
rf_pred_ros = rf_model_ros.predict(X_test_ros)
xgb_pred_ros = xgb_model_ros.predict(X_test_ros)
```

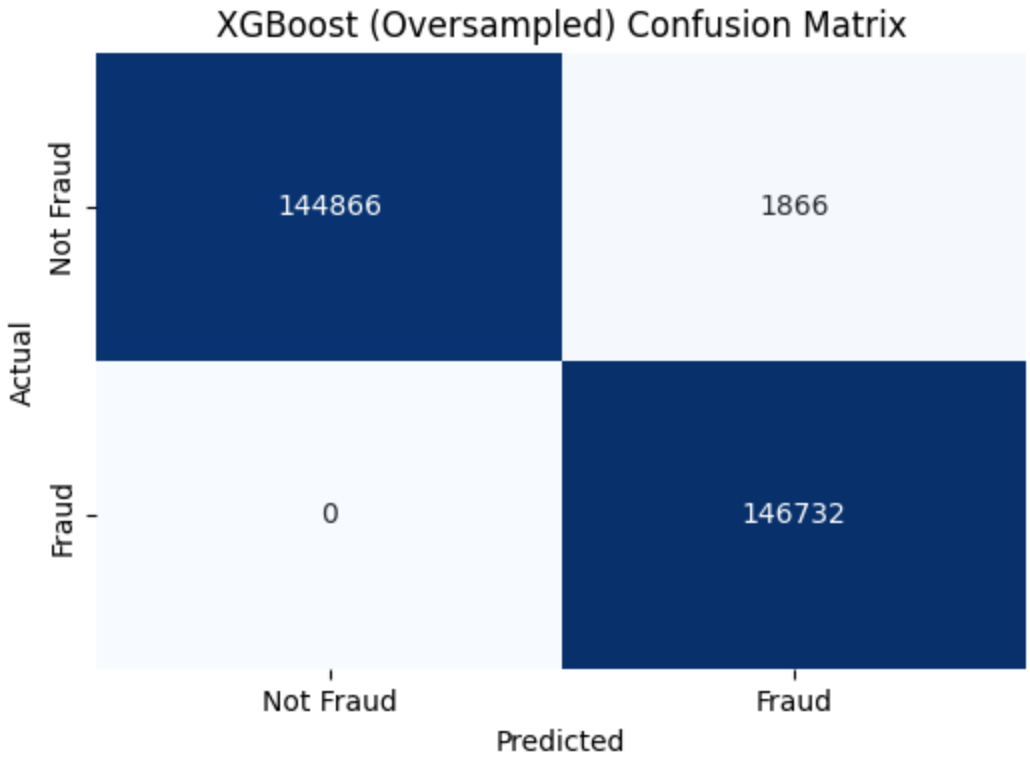
In [35]:
```
print("Random Forest (Oversampled) Confusion Matrix:")
cm_rf_ros = confusion_matrix(y_test_ros, rf_pred_ros, labels=[0,1])
draw_confusion_matrix(cm_rf_ros, title='Random Forest (Oversampled) Confusion Matri
```

```
Random Forest (Oversampled) Confusion Matrix:
```



In [36]:
```
print("XGBoost (Oversampled) Confusion Matrix:")
cm_xgb_ros = confusion_matrix(y_test_ros, xgb_pred_ros, labels=[0,1])
draw_confusion_matrix(cm_xgb_ros, title='XGBoost (Oversampled) Confusion Matrix')
```

```
XGBoost (Oversampled) Confusion Matrix:
```



In [37]:
```
scaled_data_big = scaler.fit_transform(data[['amount', 'gender_E', 'gender_F', 'gen
                                             'age_2.0', 'age_3.0', 'age_4.0', 'age_
                                             'category_1', 'category_2', 'category_
                                             'category_6', 'category_7', 'category_
                                             'category_11', 'category_12', 'categor
                                             'merchant_total_trans', 'customer_tota
                                             'category_total_trans', 'customer_ID',
                                             'amount_thresh_2', 'amount_thresh_3',
                                             'amount_thresh_6', 'amount_thresh_7']]
```

In [38]:
```
kmeans = KMeans(n_clusters=3, random_state=123, n_init=25)
kmeans_labels = kmeans.fit_predict(scaled_data_big)
kmeans_data = pd.DataFrame(scaled_data_big, columns=['amount', 'gender_E', 'gender_
                                                     'age_2.0', 'age_3.0', 'age_4.0
                                                     'category_1', 'category_2', 'c
                                                     'category_6', 'category_7', 'c
                                                     'category_11', 'category_12',
                                                     'merchant_total_trans', 'custo
                                                     'category_total_trans', 'custo
                                                     'amount_thresh_2', 'amount_thr
                                                     'amount_thresh_6', 'amount_thr
kmeans_data['cluster'] = kmeans_labels
kmeans_data['fraud'] = data['fraud']
```

In [39]:
```
print("K-means Cluster vs Fraud:")
print(pd.crosstab(kmeans_data['cluster'], kmeans_data['fraud']))
```

```
K-means Cluster vs Fraud:
fraud       F      NF
cluster
0        6920  586928
1          58       0
2         222       0
```

In [40]:
```
pca_kmeans = PCA(n_components=3)
pca_kmeans_data = pca_kmeans.fit_transform(kmeans_data.drop(['cluster', 'fraud'], a
pca_kmeans_df = pd.DataFrame(pca_kmeans_data, columns=['PC1', 'PC2', 'PC3'])
pca_kmeans_df['cluster'] = kmeans_data['cluster']
```

In [41]:
```
fig = px.scatter_3d(pca_kmeans_df, x='PC1', y='PC2', z='PC3', color='cluster', titl
fig.update_traces(marker=dict(size=3))
fig.show()
```